```
#q1
import numpy as np

temperature = np.array([25, 28, 31, 35, 38, 27, 33, 40])

temperature = temperature + 2

print("Temperature in Celsius", temperature)

fahrenheit = (9/5) * temperature + 32
print("Temperature in Fahrenheit", fahrenheit)

readings = temperature[temperature > 32]
print("Readings greater than 32", readings)

print("Number of readings greater than 32", len(readings))
```

```
Temperature in Celsius [27 30 33 37 40 29 35 42]
Temperature in Fahrenheit [ 80.6  86.   91.4  98.6 104.   84.2  95.  107.6]
Readings greater than 32 [33 37 40 35 42]
Number of readings greater than 32 5
```

```
#q2
import numpy as np

steps = np.array([[5000, 6200, 7100],
                  [8000, 7500, 9000],
                  [4500, 5100, 4800],
                  [9000, 8500, 9500]])

print("Total steps", np.sum(steps))
print("Mean steps", np.mean(steps))
print("Maximum steps", np.max(steps))
print("Minimum steps", np.min(steps))

print("Total steps for each day", np.sum(steps, axis=0))
print("Total steps for each user", np.sum(steps, axis=1))

position = np.unravel_index(np.argmax(steps), steps.shape)
print("Position of maximum steps", position)
```

```
Total steps 84200
Mean steps 7016.666666666667
Maximum steps 9500
Minimum steps 4500
Total steps for each day [26500 27300 30400]
Total steps for each user [18300 24500 14400 27000]
Position of maximum steps (np.int64(3), np.int64(2))
```

```
#q3
import numpy as np

original = np.array([1, 2, 3, 4, 5, 6])

subset = original[1:5]
subset[0] = 999

print("Original array", original)
print("Subset", subset)

copied = original[1:5].copy()
copied[0] = 500

print("Original array", original)
print("Copied array", copied)

arr = np.arange(1, 13).reshape(3, 4)

print("3 x 4 matrix")
print(arr)

print("First row", arr[0])
print("Last row", arr[-1])
print("Second column", arr[:, 1])
print("Rows 1-2 and columns 2-3", arr[0:2, 1:3])

flat = arr.flatten()
rav = arr.ravel()

print("Flatten", flat)
```

```
print("Ravel", rav)

rav[0] = 100
print("After changing ravel")
print("Original array", arr)
print("Ravel", rav)

flat[1] = 200
print("After changing flatten")
print("Original array", arr)
print("Flatten", flat)

print("Shape", arr.shape)
print("Dimensions", arr.ndim)
print("Size", arr.size)
print("Data type", arr.dtype)
```

```
Original array [  1 999   3   4   5   6]
Subset [999   3   4   5]
Original array [  1 999   3   4   5   6]
Copied array [500   3   4   5]
3 x 4 matrix
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
First row [1 2 3 4]
Last row [ 9 10 11 12]
Second column [ 2  6 10]
Rows 1-2 and columns 2-3 [[2 3]
 [6 7]]
Flatten [ 1  2  3  4  5  6  7  8  9 10 11 12]
Ravel [ 1  2  3  4  5  6  7  8  9 10 11 12]
After changing ravel
Original array [[100   2   3   4]
 [  5   6   7   8]
 [  9  10  11  12]]
Ravel [100   2   3   4   5   6   7   8   9  10  11  12]
After changing flatten
Original array [[100   2   3   4]
 [  5   6   7   8]
 [  9  10  11  12]]
Flatten [  1 200   3   4   5   6   7   8   9  10  11  12]
Shape (3, 4)
Dimensions 2
Size 12
Data type int64
```

```
#q4
import numpy as np

X = np.array([[6, 70, 3],
              [5, 50, 6],
              [8, 80, 2],
              [4, 30, 8]])

y = np.array([40, 65, 30, 85])

print("Shape of X", X.shape)
print("Dimensions of X", X.ndim)

print("Transpose of X")
print(X.T)

xtx = X.T @ X
print("X transpose X")
print(xtx)

inverse = np.linalg.inv(xtx)
print("Inverse")
print(inverse)

beta = inverse @ X.T @ y
print("Coefficients", beta)

new_user = np.array([5, 40, 7])
prediction = new_user @ beta

print("Predicted assistance score", prediction)
```

```
Shape of X (4, 3)
Dimensions of X 2
Transpose of X
[[ 6  5  8  4]
 [70 50 80 30]
 [ 3  6  2  8]]
```

```
X transpose X
[[  141  1430    96]
 [ 1430 14700   910]
 [   96   910   113]]
Inverse
[[ 1.10521428 -0.09848746 -0.14581398]
 [-0.09848746  0.00891203  0.01190129]
 [-0.14581398  0.01190129  0.0368847 ]]
Coefficients [-0.15921454  0.14083853 10.06302242]
Predicted assistance score 75.27862544779117
```

```
#q4(b)-X.T changes the rows into columns, so the 4×3 matrix becomes a 3×4 matrix.
#q4(f)-The coefficients represent the effect of sleep, activity, and stress on the predicted assistance score.
```

```
Start coding or generate with AI.
```